Robust Sensing and Chemotaxis

Joshua S. Weitz[1, 2, *]

[1] *School of Biology, Georgia Institute of Technology, Atlanta, GA, USA*
[2] *School of Physics, Georgia Institute of Technology, Atlanta, GA, USA*
(Dated: September 25, 2020)

I. TOWARD CHEMOTAXIS IN SINGLE-CELLED ORGANISMS

The goal of this lab is to develop computational tools to simulate an individual moving on a 2D surface either towards or away from a chemical signal. Such movements extend beyond random diffusion to chemotaxis, i.e., the directed motion of an organism toward or away from a chemical signal. This laboratory also serves as a bridge between the first thematic modules on molecular and cellular biosciences and the next thematic set on organismal behavior and physiology. In doing so, it is important to keep in mind: bacteria (and archaea) are organisms too!

To get all the way to simulating chemotaxis requires the translation of a more than just a few concepts; at minimum some understanding of how to represent the conversion of extracellular chemical concentrations into an intracellular signal. The next step is to combine such principles with movement dynamics. In essence, the study of chemotaxis already integrates concepts of organismal behavior and physiology, albeit at a microbial scale. Because we are focusing on *E. coli*, the movement will be centered on 'runs' and 'tumbles'. The laboratory also helps to prepare you for independent work: integrating signal transduction and movement into a unified model of chemotaxis (a subject treated in depth in the homework problems in the associated chapter in the textbook).

Notably, chemical concentration gradients may change over time. Hence, this lab will cover a set of core techniques that can bridge the gap between stochastic models of random walks to simulations of chemotaxis in single-celled organisms. The key biological concept is that individual bacteria can detect subtle changes in the concentration of chemicals. Some of these chemical may be beneficial to the bacteria and others may be detrimental. Bacteria, including *E. coli*, have evolved to change their behavior so that they preferentially move towards chemoattractants and away from chemorepellants. In a overly simplistic sense, this process of 'chemotaxis', i.e., the movement driven by chemicals, can be broken down into three core processes. First, bacteria can detect extracellular chemicals via receptor-ligand processes akin to enzyme kinetics. Second, interactions than initiate a signal transduction cascade, mediated by enzyme kinetics. Third, changes in intracellular concentrations of key proteins modulate the behavior of the bacteria, enabling it to have chemotaxis behaviors. As such the laboratory covers a few essential elements:

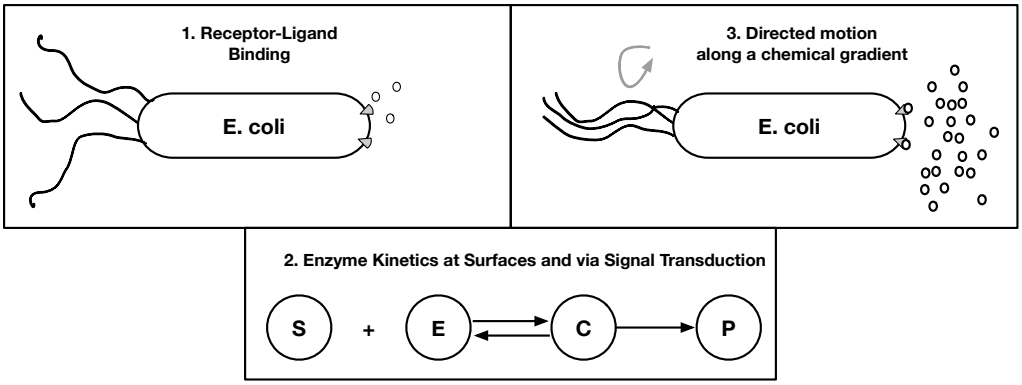


FIG. 1: Multi-part process of chemotaxis, including binding of chemical ligands to receptors, signal transduction, and change in cellular physiology and behavior.

- Basics of signal transduction, from Michaelis Menten and beyond.

*Electronic address: jsweitz@gatech.edu; URL: http://ecotheory.biology.gatech.edu

Typeset by REVTEX

- Including time-dependent forcing functions and parameters into numerical integration of coupled ordinary differential equations.
- Reviewing probability distributions and their moments.
- Building a stochastic spatial model involving diffusion
- Measuring the expected properties of diffusion.

These techniques are integrated with both the main text and associated homework. As is typical, these elements are the start of, but not the final word, on investigating how cells integrates information to change and adapt their behavior to local conditions.

II. ENZYME KINETICS

Signal transduction describes the process by which cells detect a chemical signal and then transform that external signal into an intracellular signal. Usually this process unfolds when a chemical 'ligand' binds to a trans-membrane receptor. This binding leads to a change in the chemical state of the receptor, in which the intra-cellular end of the receptor molecule then modifies a diffusible protein. This modification can be a (de)phosphorylation or a (de)methylation event. The core modeling framework that underpins signal transduction is that of chemical reaction theory. And, to model chemical reactions requires some way of representing perhaps the simplest approximation of changes in chemical concentrations: Michaelis Menten kinetics.

Consider a chemical reaction, in which a substrate S (e.g., a ligand) binds reversibly to an enzyme, E, forming a complex, C. This complex can then generate a product, P. The dynamics of the concentration of this system can be described as:

$$\dot{S} = -k_+ SE + k_- C \tag{1}$$
$$\dot{E} = -k_+ SE + (k_- + k_f)C \tag{2}$$
$$\dot{C} = k_+ SE - (k_- + k_f)C \tag{3}$$
$$\dot{P} = k_f C \tag{4}$$

Translating these equations into a computational model is the first challenge problem.

Challenge Problem: Simulation Chemical Reactions

Fill in the following dynamical function to recapitulate the core numerical integration of enzyme kinetics:

```
def mm_dyn(y,t,pars):
    """
    mm_dyn(t,y,pars):
    Simulates a full enzyme kinetics model
    """
    #Assign variables
    S, E, C, P = y[0], y[1], y[2], y[3]
    #Core dynamics
    dydt=np.zeros(4)
    dydt[0]=...
    dydt[1]=...
    dydt[2]=...
    dydt[3]=...
    return dydt
```

Solutions to Challenge Problem: Simulating Chemical Reaction
A computational model of enzyme kinetics must ensure that each of the forward and back reactions are included in the computations as follows:

```
def mm_dyn(y,t,pars):
    """
    mm_dyn(t,y,pars):
    Simulates a full enzyme kinetics model
    """
    #Assign variables
    S, E, C, P = y[0], y[1], y[2], y[3]
    #Core dynamics
    dydt=np.zeros(4)
    dydt[0]=-pars['kplus']*S*E + pars['kminus']*C
    dydt[1]=-pars['kplus']*S*E + pars['kminus']*C + pars['kf']*C
    dydt[2]= pars['kplus']*S*E - (pars['kminus']+pars['kf'])*C
    dydt[3]= pars['kf']*C
    return dydt
```

It is now time to simulate a typical reaction. Consider a ligand (the enzyme) that is present in μM concentrations, with binding rates of 0.1 $\text{sec}^{-1}\mu$ M^{-1}, unbinding rates of 100 sec^{-1}, and forward reaction rates of 1 sec^{-1}. Use the following code-base to simulate and visualize the dynamics:

```
# Parameters - uM and sec units
pars={}
pars['kplus'] = 0.1
pars['kminus'] = 100;
pars['Km'] = pars['kminus']/pars['kplus']
pars['kf']=1
pars['tf']=3600

# Simulation
t = np.linspace(0,pars['tf'])
y = integrate.odeint(mm_dyn,[10,1,0,0],t,args=(pars,))

#Plot
plt.plot(t,y[:,0], color='k', linestyle='-', linewidth=3)
plt.plot(t,y[:,3], color='k', linestyle='--',linewidth=3)
```

The resulting figure panels show the depletion of substrate followed by generation of product.

Challenge Problem: Product Dynamics

Compare and contrast the *rate* of product dynamics, i.e., dP/dt, with that anticipated via the Michaelis-Menten approximation:

$$\frac{\text{dP}}{\text{dt}} \approx k_f E_0 \frac{S(t)}{k_-/k_+ + S(t)} \tag{5}$$

In Fig. 2-right, the approximation is compared to that of the full solution. The challenge is to recapitulate this for yourself, i.e., comparing and contrasting the actual product generation rate vs. that of the approximation. If you really want an extra challenge, try to identify conditions in which the approximation breaks down.

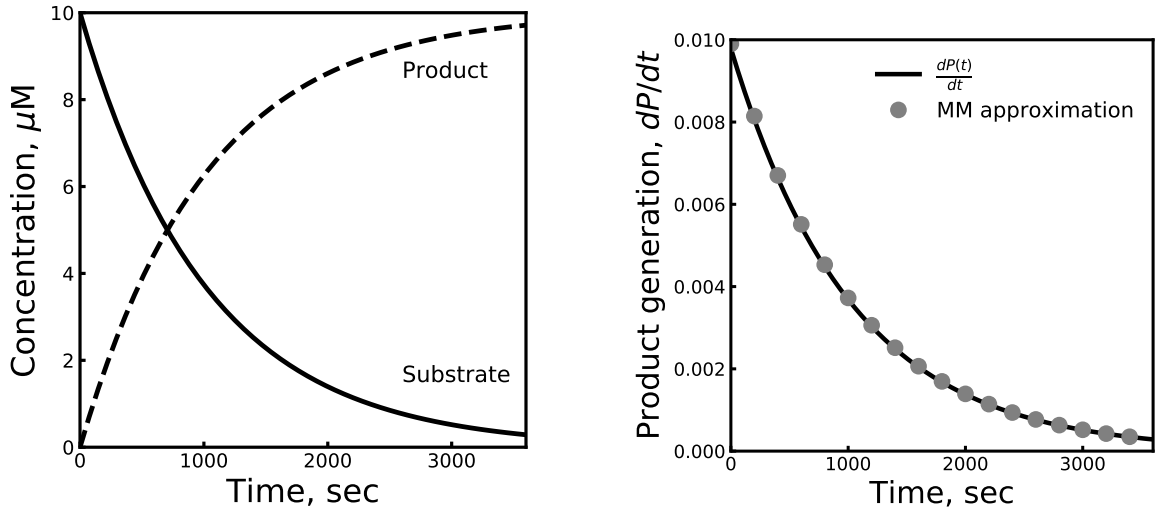


FIG. 2: Dynamics of full enzyme kinetics, including substrate and product (left) with the MM approximation (right).

Solutions to Challenge Problem: Product Dynamics The code to compare and contrast the realized derivative and the expected derivative given the MM approximation is as follows. Note that there are more sophisticated ways to do this, i.e., by recalculating or storing the instantaneous derivative during simulations. Hence, be mindful that this solution is in its own way an approximation.

```
# Simulate the dynamics
t = np.arange(0,pars['tf'],pars['dt'])
E0 = 1
y = integrate.odeint(mm_dyn,[10,E0,0,0],t,args=(pars,))

# Compare and contrast the derivatives
dpdt_real = (y[1:,3]-y[:-1,3])/pars['dt']
dpdt_approx = pars['kf']*E0*y[:-1,0]/(pars['Km']+y[:-1,0])

#Plot the comparison
plt.plot(t[:-1],dpdt_real, color='k', linestyle='-', linewidth=3)
# space out the overlay (for visualization purposes)
plt.plot(t[:-1:200],dpdt_approx[:-1:200], marker='o', linestyle='',
         markerfacecolor=[0.5,0.5,0.5], markeredgecolor=[0.5,0.5,0.5],
         markersize=10)
```

III. TIME DEPENDENT FUNCTIONS IN DIFFERENTIAL EQUATIONS

Thus far the laboratories have focused on state-dependent changes in differential equations. However, changes may also be time-dependent. As such when modeling environments with external forcing it will be essential to include time-dependent parameters and/or functions. Time-dependent parameters are easily implementable in `integrate.odeint` Recall the mRNA dynamics from a previous laboratory given simple high/low production rates and a fixed decay rate. Now, consider the following time-dependent ODE for mRNA dynamics:

$$\frac{dm}{dt} = \beta(t) - \alpha m$$

Here, the production rate of mRNA, $\beta(t)$ depends on time. Suppose the gene is turned on until $t = 10$ hrs. Previously, we would simulate two different ODEs in sequence, one with a β term and one without β. This can be simplified as

a single equation.

$$\frac{dm}{dt} = \Theta(10 - t)\beta - \alpha m$$

where we use the Heaviside function, $\Theta(x)$, which is one when $x > 0$ and zero otherwise. A similar equation can be called by `integrate.odeint`

```
def pulse_production(x,t):

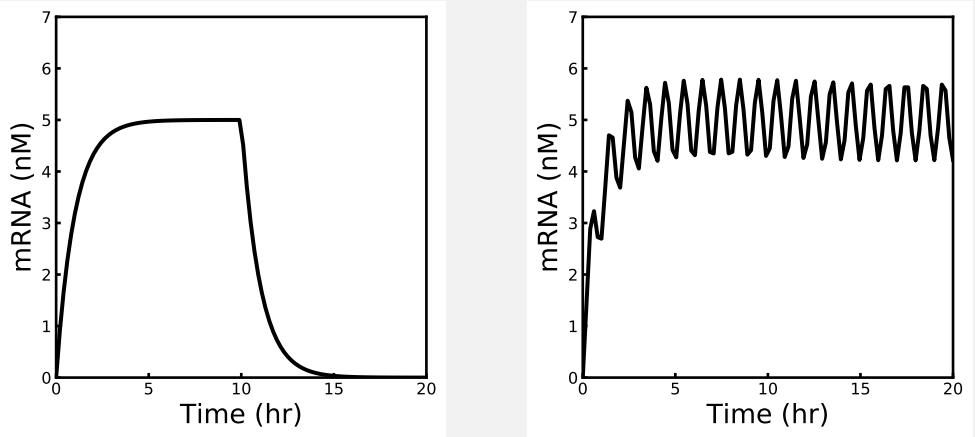
    alphaval = 1 # Decay rate, hrs^-1
    betamax = 5 # Production rate, e.g., nM/hrs

    return (t<10)*betamax - alphaval*x
```

where we defined the parameter values in the function scope. Note that parameters can be made time-dependent when defining the dynamical system.

Challenge Problem

Include the time-dependent mRNA dynamics and develop your own simulation over a 20 hr period. The resulting dynamics starting with 0 mRNA should look like the following, on the left panel:



Next, you can use more exotic functions than simple on and off switches. Change the above code such that the production rate follows $\beta(t) = (sin(2\pi t) + 1)\beta_{mid}$ with $\beta_{mid} = 5$ – this ranges from a minimum of 0 nM/hr to 10 nM/hr. You should now get oscillatory dynamics as in the above, right panel. How does the time period and the phase (where the peak is located) of the mRNA compare with the production rate?

Solution: A minimal script to simulate mRNA dynamics is as follows:

```
x0 = 0
tmax = 20
t = np.linspace(0,tmax,1000)
mrna = integrate.odeint(pulse_production, x0, t)
plt.plot(t,mrna,linewidth=3)
plt.xlabel('Time',fontsize=20)
plt.ylabel('mRNA',fontsize=20)
```

However, for the more general case, it may be worthwhile to use an anonymous function, e.g.

```
x0=0
tmax = 20
pars={}
pars['alpha']=1 # hr^-1
pars['beta_mid']=5 # nM/hr
pars['beta'] = lambda t: pars['beta_mid']*(t<10) # nM/hr
t=np.linspace(0,tmax,100)
mrna = integrate.odeint(pulse_production,x0,t,args=(pars,))
plt.plot(t,mrna,linewidth=3,color='k')
plt.xlabel('Time (hr)',fontsize=20)
plt.ylabel('mRNA (nM)',fontsize=20)
```

and then modify the dynamics code to refer to a pre-specified function.

```
def pulse_production(x,t,pars):

    return pars['beta'](t) - pars['alpha']*x
```

Note that the dynamics oscillate around the ratio of the average value of $\beta(t)/\alpha$ with a period approaching 1 hr.

IV. PROBABILITY DISTRIBUTION REDUX

Previous labs introduced the concept of the difference between an individual stochastic trajectory and the ensemble. The ensemble is a very large collection of trajectories given the same governing parameters. In the case of Markov Chains, we found that averaging the properties of many stochastic trajectories, e.g., mean and variance, coincided nearly perfectly with the predictions expected given the theoretical probability distribution obtained via the transition matrix. Here we make the link between individual samples of a probability distribution and the limiting probability distribution itself. Such a link will be important in building a model of *E. coli* movement.

A. Binomial distributions

First, suppose we're flipping 15 fair coins (equal chance heads or tails) and want to know the expected number of heads. Recall that the number of heads observed should follow a binomial distribution with $N = 15$ and $p = .5$ with an average number of heads of $7.5 = 15 \times 0.5$. However the most likely outcome should be either 7 or 8 heads in any individual 'simulation'. How can we move from the expected probability distribution to a collection of random samples? Below, we will evaluate three ways.

Theory : Use `stats.binom.pmf` to obtain the probability distribution over the discrete support `0:N`.

Sampling using custom code; As an alternative, you could sample binomial numbers from this discrete distribution using `randi_wheel` from before.

```
import numpy as np
from scipy import stats
# Function
def randi_wheel(fitness):
```

```
    tmp_weights = fitness/sum(fitness)
    tmp_order = np.cumsum(tmp_weights)
    x = np.argmax(np.random.uniform()<=tmp_order)
    return x

# Script
N=15
p=0.5
nvals = np.arange(N+1) # The actual support of the pdf
mypdf = stats.binom.pmf(nvals,N,p)
rand_bino = nvals[randi_wheel(mypdf)] # Generate one random number using mypdf
```

It is important to use the `nvals[randi_wheel(mypdf)]` rather than `randi_wheel(mypdf)`. The reason is that randomly generated numbers from a fixed pdf will return an index. Hence, the 1st index corresponds to the value of 0 in the binomial distribution, the 2nd index to 1, and so on, such that the $N+1$ index corresponds to the value N, as expected.

Built-in functions : Python has a built-in function for sampling binomially distributed random numbers, i.e., `rand_builtin = np.random.binomial(N,p)`.

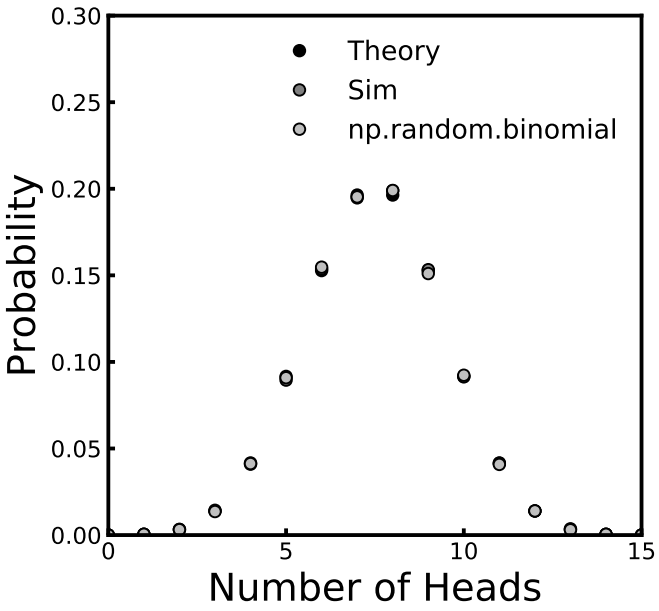
Challenge Problem: Binomial Distributions

Compare the results of sampling 10^4 different events using a customized code, a built-in Python random number generation, and the theoretical distribution for a binomial process each with $N = 15$ trials and a $p = 0.5$ probability of success.

Solutions to Challenge Problem: Binomial Distributions Consider the following script:

```
# Define the parameters and theoretical expectations
N = 15
p = 0.5
xsupp = np.arange(0,N+1)
mypdf = stats.binom.pmf(xsupp,N,p)

# Simulate the ensemble
totruns = 10**5
rand_builtin = np.random.binomial(N,p,totruns)
rand_bino = np.zeros(totruns)
for jj in range(totruns):
    rand_bino[jj]=randi_wheel(mypdf)
```

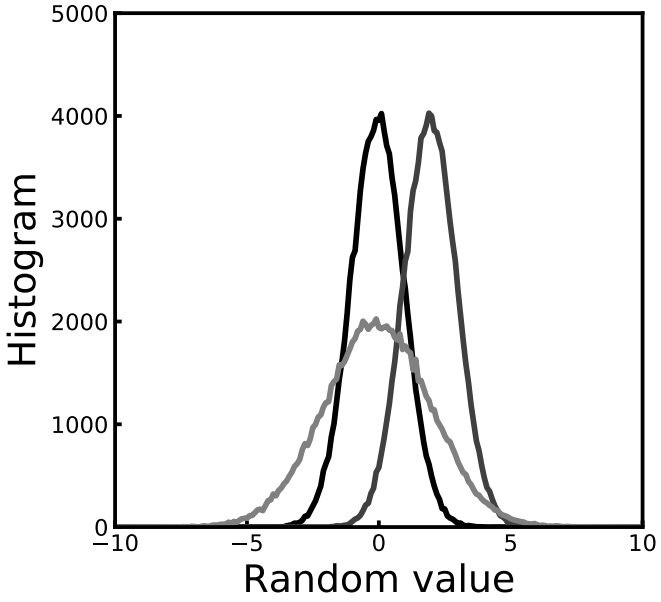


Both the built-in and customized methods of random sampling are equivalent and converge to the same theoretical distribution. Critically, this example helps illustrate the power of the `randi_wheel` function which enables random sampling from any probability distribution with finite support, i.e., whether parametric or non-parametric. Note that you could modify the customized command to generate different size matrices of output.

B. Gaussian (or normal) distributions

Notice the binomial distribution is unimodal – it has a distinct peak. There are many distributions with distinct peaks, including the normal (or Gaussian) distribution which is defined over the entire real line and peaks at the mean value μ. The width of the normal distribution is defined by its variance. The `np.random.normal` outputs normally distributed random samples. Compare histograms derived from 10^5 samples from the following code:

```
norm1 = np.random.normal(size=10**5)
norm2 = np.random.normal(size=10**5)+2
norm3 = 2*np.random.normal(size=10**5)
```

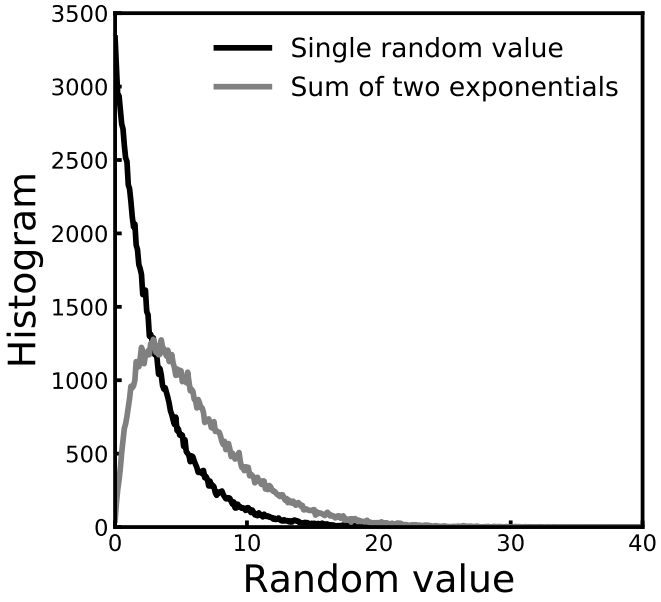


What is the mean and variance for each normal distribution? As should be evident, these three random numbers have means 0, 2 and 0 respectively (as `norm1` is shifted by a factor of 2.). However, the shifts do not effect the variance, which is defined as the expected value of the centralized moment, i.e., $E\left((X-\mu)^2\right)$ where X is a random number sampled from a distribution with mean μ. In the above cases, the first two random numbers have variances equal to that of the standard normal distribution – 1. However, `norm3` has a variance of 4, because each random number is stretched by a factor of 2 (whose expected squared deviation is then 4).

C. Gamma distributions

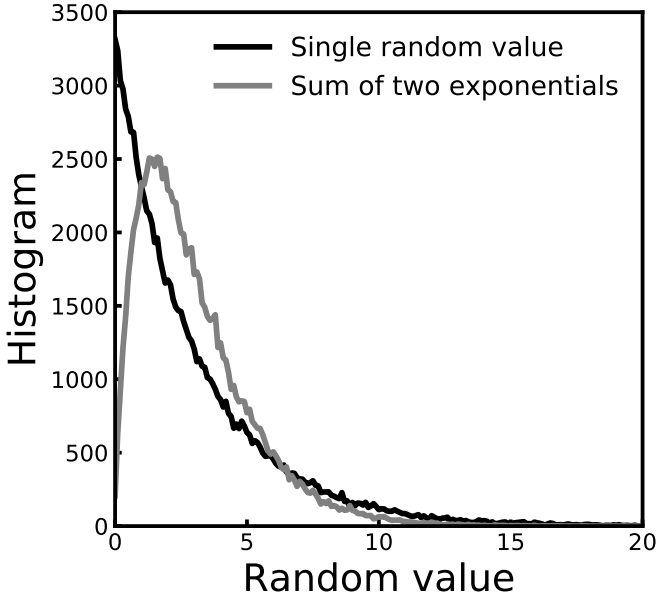
Another potentially unimodal distribution is the gamma distribution. Recall, that the time between events in a Poisson process follows the exponential distribution. But what happens if two events need to happen for a reaction to occur (e.g., multiple bindings leading to the chemical modification of a transcription factor, kinase, or methylase)? How long would it take for $n = 2$ sequential Poisson processes to both have events? Let's sample two exponentially distributed vectors and compare the distribution of one to their sums.

```
exp1 = np.random.exponential(3,size=10**5)
exp2 = np.random.exponential(3,size=10**5)
exp12 = exp1+exp2
```



The histogram for a single draw ($n = 1$) follows the exponential distribution, but the histogram for the sum of two draws ($n = 2$) is unimodal. Compare the mean and variance for each distribution. The results are fairly intuitive, they are both related by a scaling factor of n. What if instead of the same mean value, the expected times in the multi-draw problem scaled with $1/n$. Compare the prior results to the following, using this code

```
exp1 = np.random.exponential(3,size=10**5)
exp2a = np.random.exponential(1.5,size=10**5)
exp2b = np.random.exponential(1.5,size=10**5)
exp12 = exp2a+exp2b
```



The key observation is that if the rate is What are the mean and variance to these distributions? Now consider $n = 2$, $n = 3$, and $n = 10$ cases with the same scaling of the rates $\frac{r}{n}$. You will find that the distribution becomes more focused around the mean. This type of dynamics often appears in models of living systems with relatively precise control over timing. For example, models of infectious disease often include multiple intra-host stages of infection. The duration in each state may be exponentially distributed (corresponding to a single rate of progression of the disease). Hence, the overall infected lifespan is gamma-distributed. This concept also applies to viral infections of microbes in which

there is a timed process between adsorption, infection, and lysis. More generally, any developmental process with multiple step-wise phases, e.g., the cell cycle or a kinetic cascade, will tend to exhibit gamma-like distributions for the total timing of events, making it more likely that events happen close to a given time rather than immediately (as in the exponential distribution). Figure 3 illustrates this conceptually.

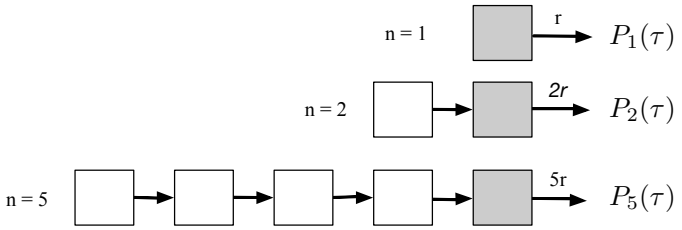
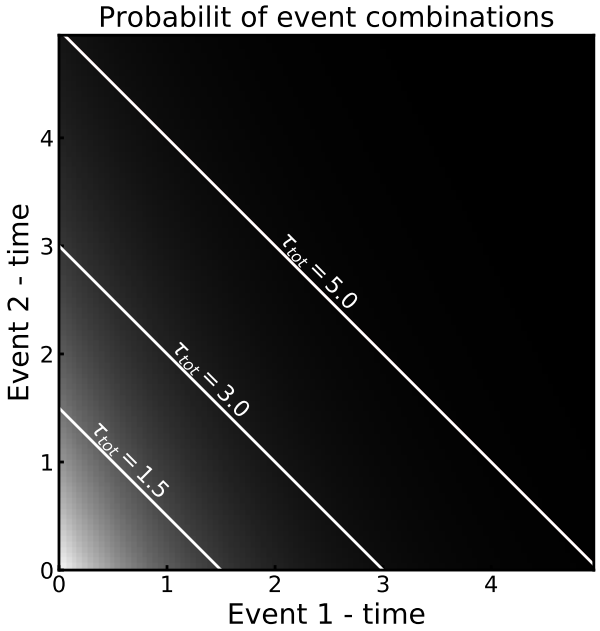


FIG. 3: Illustration of measuring the distribution of the cumulative time for n independent stochastic processes, each with an individual transition rate of rn. As shown in this section, the average time of $P_n(\tau)$ is exponentially distributed when $n = 1$ and Gamma distributed when $n > 1$, i.e., it has a unimodal peak, with ever-narrowing variance as n increases.

This set of calculations raises some questions: why does adding up many random numbers each sampled from an exponential yield something unimodal? Some insight can be gained by visualizing the set of event times, t_1 and t_2 such that $t_{tot} = t_1 + t_2$. The puzzle arises when we think about the probability of some combination of event times (t_1, t_2) when each is exponentially distributed with characteristic times τ_1 and τ_2. The intensities of the joint probability of such event times

$$p(t_1, t_2) = \frac{e^{-t_1/\tau_1} e^{-t_2/\tau_2}}{\tau_1 \tau_2} \tag{6}$$

can be seen in the variation of colormap intensities of the following figure. As you can see, the intensity is brightest at the bottom left, i.e, when both t_1 and t_2 are near zero. However, the white solid lines denote combinations of event times whose *sum* is equivalent. As you can see, although very short events are the most likely, there are very few combinations of such events (the number of combinations is proportional to the length of the solid white lines). Similarly, although there are many combinations of events in which the sum is large (long white lines in the upper right), the probability of any such event vanishes exponentially. This argument suggests that there will be almost zero probability of having two events happen at once and likely almost zero probability of having two events take a very very long time to occur. Hence, the most likely sum of event times is located at some intermediate value!



The core elements of this figure was generated through the following script (you can add your own labels and other niceties):

```
# Define the sampling
dt = 0.05
t1 = np.arange(0,5,dt)
t2 = np.arange(0,5,dt)
tau1 = 1
tau2 = 2

# Define the joint probabilities
t1_g, t2_g = np.meshgrid(t1,t2)
p1 = np.exp(-t1/tau1)/tau1
p2 = np.exp(-t2/tau2)/tau2
ptot = np.exp(-t1_g/tau1)*np.exp(-t2_g/tau2)

# Plot an equal total time to set the axis
# recalling that the implicit dt of the grid is 0.05
fig = plt.figure()
ax = fig.gca()
plt.plot(t1/dt,100-t1/dt,color='r')
# Overlay the image
ax.imshow(ptot)
# Plot different total times
plt.plot(t1*20, 100-t1*20, color='w',linewidth=2)
plt.plot(t1*20, 60-t1*20, color='w',linewidth=2)
plt.plot(t1*20,30-t1*20, color='w',linewidth=2)

# Because of the meshgrid command the size should
# be equal to the grid, and not the actual time (which
# varies between 0 and 5 for both t1 and t2)
plt.xlim([0,100])
plt.ylim([0,100])
```

V. E. COLI MOVEMENT

Finally, *E. coli* time! As discussed in class, E. coli move by running and tumbling. For short distances, "running" involves rotating the flagella in order for the bacterium to move in a straight line at a constant velocity. Tumbling involves rotating the flagella in the opposite direction in order to stop and re-orient before running again.

We will develop a model that tracks the cell's x-y position over time as it randomly runs and tumbles. Initially, the bacterium will run and tumble in an environment free from chemical signals. That is, the switching between running and tumbling occurs as a Poisson process with rate λ sec^{-1}. The bacterium runs at a constant speed between tumbles at a speed, v in units of μm/sec, given its current orientation θ. Thus, after a run the x-position updates $x(t+\Delta t) = x(t) + dx = x(t) + v\cos\theta\Delta t$, similarly the y-position updates $y(t+\Delta t) = y(t) + dy = y(t) + v\sin\theta\Delta t$. Here, Δt is the time between runs as sampled from the Poisson process. For the time being let's assume tumbles occur instantaneously and result in a new randomly selected orientation, θ. We can use the Gillespie algorithm to simulate the stochastic movement dynamics. The following code provides initial values for v and λ and you are free to modify them to reflect the values for *E. coli* moving through rich media, e.g., $v \approx 30$ μm/sec and $\lambda \approx 1$ sec^{-1}.

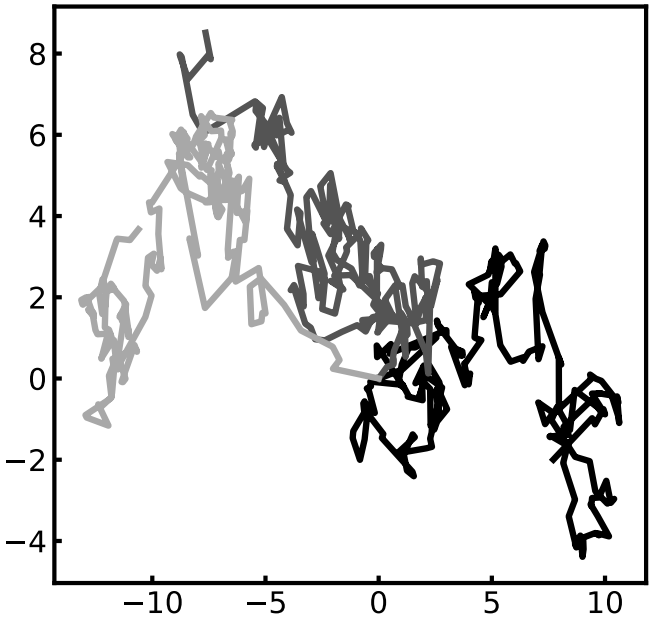
```
# Parameters and constraints
tmax = 100 # seconds
v = 1 # microns/sec
currangle = np.random.uniform()*2*np.pi
tumblerate = 2 # per sec

# Initialize the system
currt = 0
currx = 0
curry = 0
X = [[currx, curry]]
t = [0]
```

```
ind = 0

# Run and tumble
while currt<tmax:
    ind = ind+1
    dt = -(1/tumblerate)*np.log(np.random.uniform()) # Find the next tumble time
    dx = v*np.cos(currangle)*dt #Advance in the x-direction
    dy = v*np.sin(currangle)*dt #Advance in the y-direction
    currt = currt + dt
    currx = currx + dx
    curry = curry + dy
    t.append(currt)
    X.append([currx,curry])
    currangle = np.random.uniform()*2*np.pi
# convert X into a numpy array
X = np.array(X)
t = np.array(t)
```

Next, plot the x-position versus the y-position over the dynamics. Here are 3 representative curves, which you should aim to reproduce:

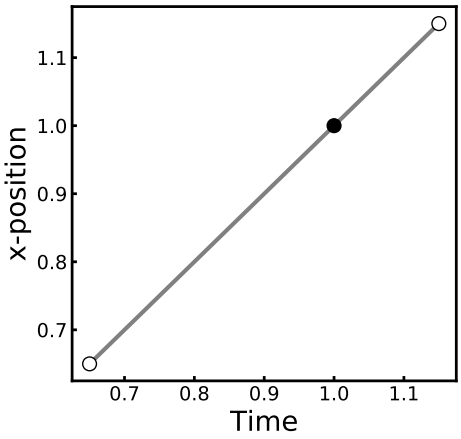


The black dot marks the origin and the initial condition of all the runs. Notice, these trajectories moved in different directions. In these plots its hard to address questions related to the timing of movement. For example, how far from the origin is an average bacterium at $t = 50$?

To compare dynamics across an ensemble, we will sample each trajectory at fixed times like we did in previous labs. However, the process itself involves jumps between new run events. As a consequence, each trajectory need not record precisely the same fixed times. Instead, we need to identify where the bacterium was at a common set of times. This is an interpolation problem. Consider two points of the x-position, x_a and x_b, sampled at times t_a and t_b, respectively, that *bracket* some sample time t_s. That is to say, what is $x(t_s)$ for $t_a \leq t_s \leq t_b$ if we know $x(t_a)$ and $x(t_b)$? We can solve for the x-position at $t = t_s$ by linearly interpolating

$$x(t_s) = x_a + \overbrace{\frac{x_b - x_a}{t_b - t_a}}^{\text{rise over run}} \overbrace{(t_s - t_a)}^{\text{elapsed from prior time}} \tag{7}$$

. Graphically, you can see below how doing so interpolates at the sample time $t_s = 1$ which lies between t_a and t_b:



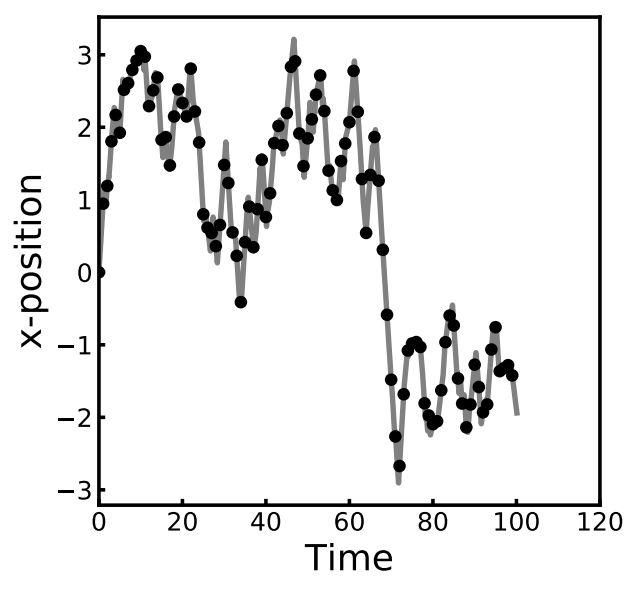
as obtained from the following code

```
lowdyn = t<1
highdyn = t>1
tlow = t[lowdyn]
ta = tlow[-1]
thigh = t[highdyn]
tb = thigh[0]
xlow = X[highdyn,0]
xa = xlow[-1]
xhigh = X[highdyn,0]
xb = xhigh[0]
deltat = tb-ta
deltax = xb-xa
slope = deltax/deltat
xnew = xa+slope*(1-ta)
```

There is a caveat. Interpolating sparsely can lead to the generation of time series that are not representive of actual paths. Use with caution in selecting an appropriate interpolation time depending on the dynamics of the system at hand.

Challenge Problem: Interpolating Stochastic Trajectories

Adapt the interpolation code above to create a function converts the stochastic trajectory of positions corresponding to tumbling times to a stochastic trajectory of x and y positions at fixed sample times (hint: loop through all the times you want to record). Confirm that you properly interpolated by comparing the sampled trajectory to the full trajectory. Here is what the answers should look like if correctly sampled in one-dimension:



Hint: There are many ways to accomplish this. The simplest way is to iterate through and interpolate each point. A more efficient way would be to keep track of prior interpolations. However, for the sake of simplicity, I recommend taking the approach which is easier to parse. There is one 'trick' to keep in mind – which is that there is not necessarily a guarantee that the sampling points coincide with the range of the measured points, so your code might crash unless you figure out how to pad the start/finish as needed.

Solutions to Challenge Problem: Interpolating Stochastic Trajectories

The following code interpolates stochastic trajectories. It is but one possible resolution, and most of the work

```
def interp_2D(t,Pos,samplet):
    '''function [samplePos] = interp_2D(t,Pos,samplet)
    Returns sample values for a 2D data in 'pos' given events at time t
    with values in samplePos which will be resampled at the time values samplet'''
    samplePos = np.zeros(shape = (len(samplet),2))
    # Earlier sample points will be set to the initial value
    tmpi = np.where(samplet <= t[0])[0]
    if len(tmpi)>0:
        samplePos[tmpi,0] = Pos[0,0]
        samplePos[tmpi,1] = Pos[0,1]
        indmin = np.where(samplet>t[0])[0][0]
    else:
        indmin = 1
    # Later points will be sent to the final value
    tmpi = np.where(samplet>t[-1])[0]
    if len(tmpi)>0:
        samplePos[tmpi,0]=Pos[-1,0]
        samplePos[tmpi,1]=Pos[-1,1]
        indmax = np.where(samplet>t[-1])[0][0]
    else:
        indmax = len(samplet)

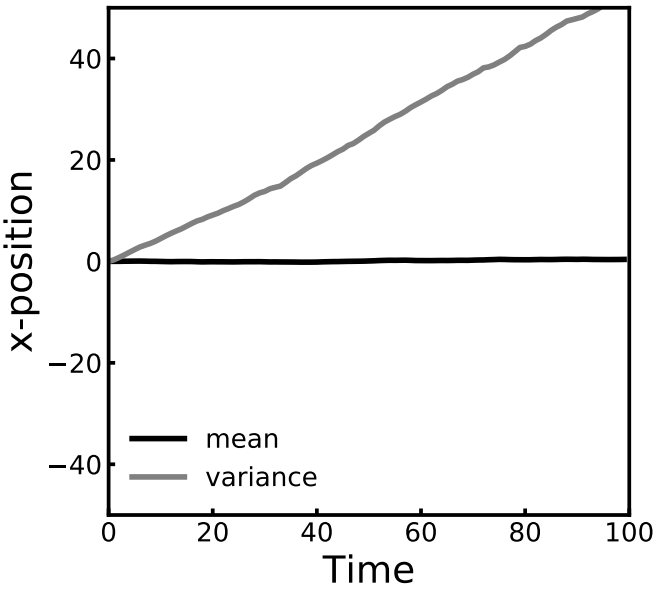
    # Resample the trajectory
    for jj in range(indmin,indmax):
        # Define the times
        lowdyn = t<=samplet[jj]
        highdyn = t>samplet[jj]
        tlow = t[lowdyn]
        ta = tlow[-1]
        thigh = t[highdyn]
        tb = thigh[0]

        # Define the values
        xlow = Pos[lowdyn,0]
        xa = xlow[-1]
        xhigh = Pos[highdyn,0]
        xb = xhigh[0]
        ylow = Pos[lowdyn,1]
        ya = ylow[-1]
        yhigh = Pos[highdyn,1]
        yb = yhigh[0]

        # Interpolate
        deltat = tb-ta
        deltax = xb-xa
        slopex = deltax/deltat
        xnew = xa+slopex*(samplet[jj]-ta)
        deltay = yb-ya
        slopey = deltay/deltat
        ynew = ya+slopey*(samplet[jj]-ta)
        samplePos[jj,0]=xnew
        samplePos[jj,1]=ynew
    return samplePos
```

Run the simulations 1000 times saving the sampled trajectories. Plot the mean and variance of the x-position over

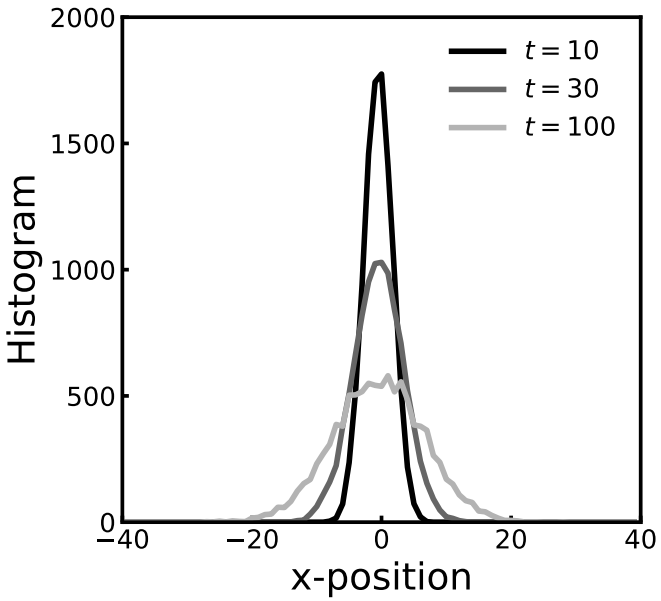
time, it should look something like the following.



Although plotting a flat line doesn't seem instructive, in this case it is meant to emphasize a key point. The execpted mean value of the x-position remains the same as the initial condition, e.g., $x = 0$. In contrast, the variance grows linearly over time. Note, in this interpretation the variance $\sigma^2 = E(x^2) - E(x)^2 = E(x^2)$ is the squared distance. Thus on average, the distance a bacterium is from the origin grows as the *square root of time*. This is not an effective means of traversing long distances, but can be efficient over small distances.

This kind of simulated movement is representative of the physical process of diffusion. This theory is appropriate for particles being thermally bombarded by smaller particles. This is the source of the Brownian motion first identified from pollen particles moving under a microscope. Given Brownian motion, the variance grows linearly over time proportional to the diffusion constant D. Indeed, it is possible to estimate the diffusion constant by fitting a line to the variance dynamics. The diffusion constant has units of length^2 time^{-1}. The dynamics of movement are based on a few parameters: velocity (length time^{-1}), tumbling frequency (time^{-1}), angle of movement (unitless). Note that via dimensional analysis we expect that the realized diffusion constant should be generated by microscopic mechanisms whose combination has the same units as D. In this case, there is only one way to construct such a value: $D = \frac{\text{velocity}^2}{\text{tumbling frequency}}$. In practice, additional modifications to D are influenced by the typical change in angle in each run, a dimensionless constant, but this is a relatively modest effect compared to the influence of the velocity and tumbling frequency. How does this value compare to your estimate of the slope?

Finally, let's explore how the distribution of *x-positions* evolves over time. Plot the distributions of x-positions for $t = 10$, $t = 30$, and $t = 100$.



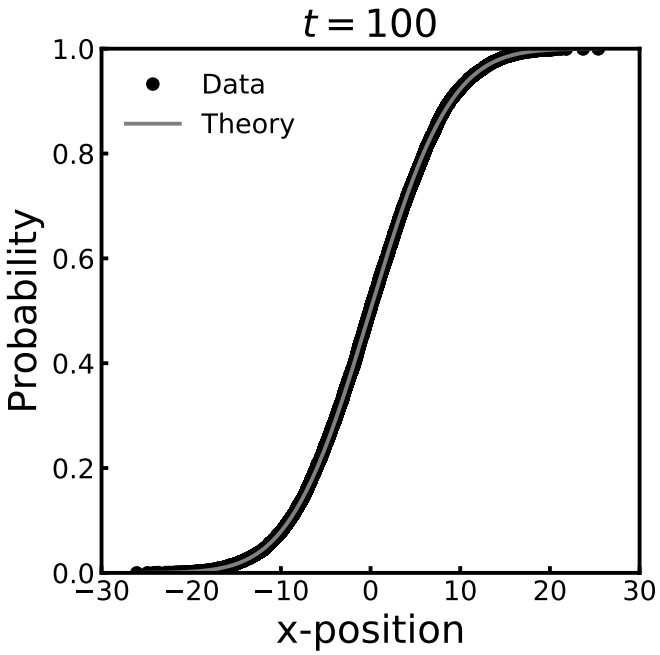
We can visually see the variance growing and the mean staying fairly constant. The distributions are fairly noisy for 1000 trajectories. The distributions look similar to normal distributions. Let's compare the cumulative distribution functions (cdf-s) of the data and of a normal distribution with the same mean and variance of the data. Use the `normcdf` matlab command to generate the theoretical cdf. Note, it takes the standard deviation not the variance as input.

```
# Calculate statistics
meandata = np.mean(xdist100)
vardata = np.var(xdist100)
sortxpos = np.sort(xdist100)
mycdf = np.arange(1,len(sortxpos)+1)/len(sortxpos)
normtheory = stats.norm.cdf(sortxpos,meandata,np.sqrt(vardata))

# Plot and compare the cdf to theory
fig = plt.figure(figsize=(5,5))
ax = fig.gca()
plt.plot(sortxpos,mycdf,linestyle='',marker='o',
         markerfacecolor='k',markeredgecolor='k')
plt.plot(sortxpos,normtheory,linewidth=2,color=[0.5,0.5,0.5])

# Make the figure look nice
plt.xlabel('x-position',fontsize=20)
plt.ylabel('Probability',fontsize=20)
plt.title(r'$t=100$',fontsize=20)
plt.legend(['Data','Theory'],loc='upper left',frameon=False,fontsize=14)
plt.setp(ax.spines.values(),linewidth=2)
ax.tick_params(labelsize=14,direction='in',width=2)
```

where `xdist100` is the data at the focal time of interest. The distribution evolves as a normal distribution over time; as is evident given the overlay of the theoretical CDF and simulation CDF for a normal distribution.



In closing, this baseline model of *E. coli* movement provides a comparison for the actual dynamics described in the textbook and developed further in the homework assignments. The homework enables you to develop model in which the movement of a particular cell depends on the local chemical state and even more importantly: the chemical gradient. Integrating internal states, external stimuli, and motion is a key step to building a quantitative model of an integrative cellular system.